

Nghiên cứu về khả năng học chính xác thuật toán của mạng nơ-ron

Báo cáo tiến độ giữa kỳ

Bài báo: *Learning to Add, Multiply, and Execute Algorithmic Instructions Exactly with Neural Networks*

Nhóm 11

23127006 – Trần Nguyễn Khải Luân

23127179 – Nguyễn Bảo Duy

23127333 – Trương Quốc Cường

23127539 – Nguyễn Thanh Tiến

Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM

Giảng viên hướng dẫn: Lê Nhựt Nam

Ngày 7 tháng 1 năm 2026

Nội dung trình bày

Phần 1: Giới thiệu

1.1. Bức tranh hiện tại: Học sâu trên miền liên tục

Nhận dạng mẫu (Pattern Recognition)

Học sâu đã rất thành công trong:

- **Thị giác máy tính:** Phân loại ảnh, nhận diện vật thể.
- **Xử lý ngôn ngữ tự nhiên:** Dịch máy, phân tích cảm xúc.

Lý do thành công (trên miền liên tục):

- Dữ liệu là tín hiệu **liên tục, trơn** (continuous, smooth).
- Mục tiêu: **xấp xỉ hàm liên tục** từ dữ liệu mẫu bằng thuật toán Gradient Descent.

1.2. Bức tranh hiện tại: Khoảng trống suy luận (Reasoning)

Nhu cầu của trí tuệ nhân tạo thể hệ mới

Các ứng dụng hiện đại (Lập trình, Toán học) đòi hỏi **suy luận** – thực thi quy tắc thuật toán, không chỉ nhận dạng mẫu.

Hạn chế của mô hình ngôn ngữ lớn hiện tại:

- Sai các phép toán đơn giản với số lớn.
- Lập luận logic không nhất quán.
- Không duy trì được quy tắc trên chuỗi dài (vòng lặp, đệ quy).

→ *Mạng chủ yếu học “dự đoán xác suất” chứ không nắm chắc “quy tắc logic”.*

Hiện nay, RNN và Transformer đã được chứng minh là **có khả năng biểu diễn (expressibility)** mọi loại lệnh và thuật toán phức tạp. Tuy nhiên, việc **Gradient Descent có khả năng học (learnability) đúng bộ trọng số đó hay không** lại là một câu chuyện hoàn toàn khác.

Đây chính là **khoảng trống nghiên cứu** đặt nền tảng cho bài báo.

1.3. Mục tiêu bài toán

Câu hỏi chính:

*Mạng nơ-ron, khi huấn luyện bằng Gradient Descent trong chế độ NTK, có thể học **đúng hoàn toàn** các thuật toán rời rạc (cộng, nhân, ...) hay không?*

Trọng tâm:

- Không bàn về “có biểu diễn được hay không”.
- Chỉ hỏi: “có **học ra đúng** thuật toán hay không”.
- Hướng tới trí tuệ nhân tạo (AI) thực sự **thực thi quy tắc**, không chỉ bắt chước dữ liệu.

1.4. Ý nghĩa khoa học

❶ Làm rõ khả năng của Gradient Descent trong việc học hàm rời rạc

- Chỉ ra rằng dù tối ưu diễn ra trong miền liên tục, GD vẫn có thể học chính xác các quy tắc rời rạc (bitwise functions) khi được biểu diễn bằng template.
- Phân tích động lực học của GD bằng NTK, chứng minh điều kiện để dự đoán hội tụ về đáp án rời rạc.

❷ Chuyển trọng tâm từ “biểu diễn được” sang “học được”

- Các kết quả Turing-complete trước đây chỉ nói về *expressivity*; bài báo này chứng minh *learnability*, mạng thật sự học được thuật toán từ dữ liệu.

❸ Nhấn tới tính đúng tuyệt đối (exact execution)

- Mục tiêu không chỉ là xấp xỉ hay đúng với xác suất cao.
- Chứng minh một mạng hai lớp có thể thực thi thuật toán **chính xác 100%** khi lập block đã học.

1.5. Ý nghĩa thực tiễn

- **Đặt nền móng cho các mô hình AI có thể học thuật toán đáng tin cậy** Cung cấp khung lý thuyết cho việc đề mạng nơ-ron học chính xác các bước thuật toán, thay vì chỉ bắt chước mẫu dữ liệu.
- **Hỗ trợ phát triển hệ thống AI yêu cầu độ chính xác cao** Các lĩnh vực như kiểm chứng, mật mã, phân tích chương trình, hay hệ thống an toàn có thể hưởng lợi từ khả năng học quy tắc chính xác ở mức bit.
- **Gợi ý hướng xây dựng mô hình giải toán / lập luận mạnh hơn** Bằng cách hiểu điều kiện để mạng học được quy trình rời rạc, nghiên cứu tương lai có thể thiết kế mô hình tổng quát hoá tốt hơn cho reasoning.

Phần 2: Kiến thức cơ sở

2.1. Mâu thuẫn: Miền liên tục vs. Miền rời rạc

Ý tưởng cốt lõi

Miền liên tục và miền rời rạc khác nhau ở **mức độ nhạy cảm với sai số**.

Trong bài toán liên tục: Sai số nhỏ thường không gây vấn đề lớn. Ví dụ: $0.99 \approx 1.00$

Trong bài toán rời rạc: Chỉ cần sai **một bit nhớ** là toàn bộ kết quả hỏng hoàn toàn — không còn khái niệm “gần đúng”. Ví dụ:

$$\begin{array}{rcccccc} & 1 & 1 & 1 & 1 & 1 & 1 \\ + & 0 & 0 & 0 & 0 & 0 & 1 \\ \hline 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$$

Hệ quả

Gradient Descent — vốn dựa trên **tính trơn (smoothness)** — gặp khó khăn khi phải học các hàm rời rạc có độ nhạy cao với sai số.

2.2. Công cụ phân tích: Neural Tangent Kernel (NTK)

Neural Tangent Kernel (NTK)

- NTK mô tả hành vi học của mạng neural khi **chiều rộng tiến đến vô hạn**.
- Trong miền (regime) này, mạng hành xử như một **mô hình tuyến tính** trong một không gian đặc trưng cố định.
- NTK cho phép *phân tích chính xác*: mạng sẽ học ra hàm gì sau khi train.

Ý nghĩa đối với bài báo

Bài báo dùng NTK để *chứng minh lý thuyết learnability*, nên NTK đóng vai trò là công cụ toán học để mô hình hoá chính xác quá trình học của mạng.

2.3. Đặc điểm thuật toán số học

Các thuật toán số học cổ điển (cộng, nhân nhị phân) có những tính chất nổi bật:

- **Cục bộ (Local):** Mỗi bước chỉ thao tác lên một phần rất nhỏ của trạng thái (ví dụ: một cặp bit và bit nhớ). Điều này cho phép mô tả thuật toán dưới dạng các *quy tắc local*.
- **Lặp lại (Iterative):** Cùng một quy tắc được áp dụng lặp đi lặp lại cho từng vị trí bit. Đây là dạng tính toán “template” - một bước mẫu được nhân bản theo chiều dài chuỗi.
- **Độc lập theo vị trí (Position-wise independence):** Quy tắc xử lý tại vị trí i không phụ thuộc vào giá trị tại các vị trí xa, mà chỉ phụ thuộc vào trạng thái rất gần (bit bên cạnh hoặc carry). Điều này giúp tách thuật toán thành các bước nhỏ có thể học độc lập.

⇒ **Bài báo khai thác chính 3 đặc tính này để xây dựng local templates, giúp mạng nơ-ron học từng bước của thuật toán một cách rõ ràng và hiệu quả.**

2.4. Tính đầy đủ Turing (Turing-completeness)

Định nghĩa

Một hệ thống được gọi là **Turing-complete** nếu nó có khả năng mô phỏng **mọi thuật toán khả tính**, tương đương sức mạnh của một máy Turing.

Để đạt tính đầy đủ Turing, một tập lệnh hoặc mô hình tính toán cần thỏa mãn ba năng lực cốt lõi:

① Thao tác bộ nhớ (Memory Manipulation)

- Có khả năng đọc, ghi và cập nhật trạng thái bộ nhớ.
- Ví dụ: Gán giá trị, tăng/giảm biến: $x = x + 1$.

② Logic điều kiện (Conditional Logic)

- Cho phép ra quyết định dựa trên trạng thái hiện thời.
- Ví dụ: Kiểm tra dấu hoặc so sánh: nếu $x < 0$ thì nhảy đến nhãn A.

③ Điều khiển luồng (Control Flow / Looping)

- Hỗ trợ lặp hoặc nhảy có điều kiện, cho phép chạy chương trình với độ dài tùy ý.
- Ý nghĩa: Cơ chế LOOP/GOTO giúp mở rộng tính toán trên dữ liệu không giới hạn.

Ví dụ về Turing-complete

Lệnh Subtract and Branch if Negative (SBN)

SBN x , y , L thực hiện:

$x = x - y$; nếu $x < 0$ thì nhảy đến L .

Dù chỉ có 1 loại lệnh duy nhất, nó vẫn đủ 3 thành phần:

- Trừ $\Rightarrow$ cập nhật trạng thái
- Kiểm tra âm $\Rightarrow$ điều kiện
- Nhảy $\Rightarrow$ điều khiển luồng

$\Rightarrow$ **SBN một mình đã Turing-complete.**

Phần 3: Phát biểu bài toán

3.1. Mục tiêu tổng quát

Mục tiêu của bài toán:

Câu hỏi trung tâm

Liệu một mạng nơ-ron kết nối đầy đủ hai tầng, được huấn luyện bằng **gradient descent trong chế độ NTK**, có thể học và thực thi **chính xác tuyệt đối** các thuật toán rời rạc ở cấp độ bit hay không?

Các thuật toán được xét:

- **Hoán vị nhị phân** — chuyển vị trí các bit theo quy tắc cố định.
- **Cộng nhị phân** — cộng hai số có lan truyền bit nhớ (carry).
- **Nhân nhị phân** — lặp lồng nhau theo từng bit (nested loops).
- **Lệnh SBN** — một lệnh Turing-complete, mô phỏng mọi thuật toán qua lặp lại.

3.2. Phương hướng chứng minh

❶ Thuật toán được phân rã thành các template cục bộ

Chuẩn hoá thuật toán thành các bước xử lý cục bộ trên trạng thái nhị phân (template $X \rightarrow Y$).

❷ Tập dữ liệu huấn luyện trực giao

Xây dựng tập dữ liệu huấn luyện được mã hoá trực giao theo từng block, bảo đảm mỗi template được học độc lập (tính học được cục bộ).

❸ Phân tích NTK

Sử dụng NTK để chứng minh gradient descent trên mạng hai tầng học đúng ánh xạ của từng template với khoảng cách an toàn (margin) > 0 .

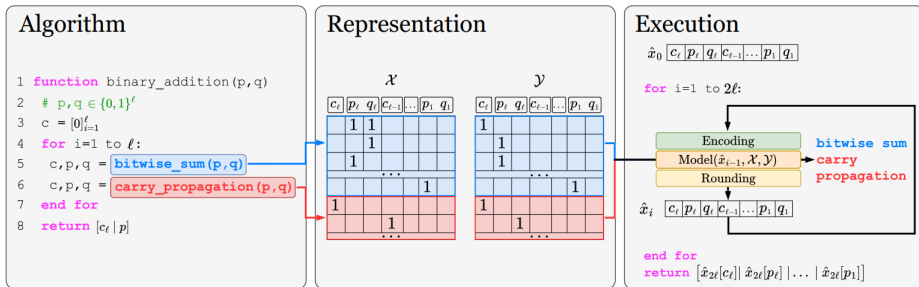
❹ Thực thi bằng vòng lặp

Lặp lại các block / template để thực thi các thuật toán dài (nhiều bước, nhiều vị trí bit).

❺ Mở rộng tới tính đầy đủ Turing (SBN)

Chứng minh các template SBN học được mà SBN turing-complete $\Rightarrow$ khung đề xuất đủ năng lực học mọi hàm có thể tính được.

Ví dụ minh họa



- **Algorithm (trái):** Thuật toán cộng được tách thành hai bước mẫu (template): *bitwise_sum* và *carry_propagation*.
- **Representation (giữa):** Mỗi bước mẫu được mã hóa thành cặp $(X \rightarrow Y)$, trong đó: X là trạng thái đầu vào (p, q, carry) , Y là đầu ra đúng của bước đó.
- **Execution (phải):** Ở bước i , mô hình nhận $\hat{x}_{i-1}$, dự đoán đầu ra, làm tròn để thu được $\hat{x}_i$, và lặp lại để hoàn thành thuật toán.

3.3. Đặc tả hình thức: Đầu vào (Input)

Đầu vào là **trạng thái thuật toán** tại một bước thực thi bất kỳ:

- Các bit của toán hạng thứ nhất
- Các bit của toán hạng thứ hai
- Bit nhớ
- Chỉ số vị trí đang xét (index/step)
- Các bit trạng thái phụ thuộc thuật toán

Ký hiệu:

$$x \in \{0, 1\}^d$$

với d là tổng số bit mã hóa trạng thái.

3.4. Đặc tả hình thức: Đầu ra (Output)

Đầu ra là vector nhị phân biểu diễn:

- Bit kết quả tại vị trí hiện tại
- Trạng thái mới (bit nhớ)
- Tín hiệu lặp cho bước tiếp theo

Ký hiệu hình thức:

$$y \in \{0, 1\}^k$$

Mỗi bước: đầu ra được làm tròn để cập nhật trạng thái:

$$x_{t+1} = \text{round}(f_{\theta}(x_t))$$

Phần 4: Công trình liên quan

4.1. Các công trình liên quan

Hướng tiếp cận	Phương pháp (ngắn gọn)	Ưu điểm	Nhược điểm
Siegelmann & Sontag (1995) – RNN Turing-complete	Chứng minh RNN có thể mô phỏng mọi thuật toán	Sức mạnh biểu đạt rất lớn $\Rightarrow$ RNN có khả năng biểu đạt mọi quy tắc thuật toán.	Không đề cập đến khả năng học bằng gradient descent
Kaiser & Sutskever (2016) – Neural GPU	Thiết kế Neural GPU để học cộng & nhân nhị phân bằng cập nhật song song.	Học được cộng/nhân với khả năng tổng quát hoá trên chuỗi dài.	Kết quả mang tính thực nghiệm; không có bảo đảm lý thuyết về học chính xác.
Pérez et al. (2021) – Transformer Turing-complete	Chứng minh Transformer có thể mô phỏng máy Turing khi cấu hình đúng.	Cho thấy Transformer cũng có khả năng biểu diễn thuật toán mạnh.	Không phân tích học bằng gradient; không đảm bảo học chính xác thuật toán ở mức bit.

4.2. Đóng góp chính của công trình

Công trình này đưa ra một kết quả mới và quan trọng:

Điểm đột phá

Lần đầu tiên chứng minh được rằng một mạng nơ-ron hai tầng, khi được huấn luyện bằng gradient descent, có thể học *chính xác* từng bước của một thuật toán rời rạc. Kết quả đạt được nhờ cách biểu diễn bằng template và phân tích trong khuôn khổ NTK.

Điểm này giúp giải quyết một hạn chế chung của các hướng nghiên cứu trước đó: mặc dù mô hình có thể biểu diễn thuật toán, nhưng *không có bảo đảm* rằng quá trình học sẽ tìm ra đúng thuật toán.

Kết quả của bài báo thu hẹp khoảng cách giữa:

- **Expressivity**: khả năng mạng nơ-ron biểu diễn thuật toán, và
- **Learnability**: khả năng học được thuật toán thông qua Gradient Descent.

**Cảm ơn Thầy và các bạn đã lắng
nghe!**